CASE STUDY 5: USER ACCOUNTS, EMAIL VERIFICATION AND CONTACT DETAILS

Q1. Identification of Invariants (6 Marks)

An invariant is a condition or business rule that must always remain true for an object to be considered valid.

In a user account management system, invariants are important because they prevent invalid user information, incorrect verification and unauthorized access.

The following are the important business invariants:

1. Email Address Must Be Valid

Every user must register with a properly formatted email address. The email address should contain the required parts, such as a valid username and domain.

This rule is important because the email address is used for communication and account verification. If an invalid email address is accepted, verification messages may not reach the user and account registration may fail.

2. Verified Email Must Have a Verification Date

If an email address is marked as verified, the system must also maintain the date and time when verification was completed.

This rule is important because the company needs proof that the user has successfully verified their email address. If a user is marked as verified without a verification date, the account information becomes inconsistent.

3. Verification Token Must Not Be Expired

A verification token should be accepted only before its expiry date and time. Once the token expires, it should no longer be valid.

This rule prevents unauthorized users from verifying accounts using old tokens. If expired tokens are accepted, attackers may misuse old verification links.

4. Phone Number Must Follow a Standard Format

Phone numbers should be stored in a consistent format, including the appropriate country code.

For example, the same phone number should not be stored in multiple different formats.

This rule is important because it helps the system validate phone numbers and send messages correctly. If different formats are used, duplicate numbers and communication problems may occur.

5. Postal Address Must Contain Complete Information

A postal address should contain the required information, such as street, city, postal code and country.

This rule is important because the company may need the address for billing, communication or service-related purposes. Incomplete addresses may result in delivery failures and incorrect customer records.

6. Suspended and Deleted Accounts Must Not Allow Login

A suspended or deleted user account should not be allowed to access the application.

This rule protects the application from unauthorized access. If suspended or deleted users can still log in, the company may face security and privacy problems.

7. Email Address Must Be Unique

Every registered user must have a unique email address. The system should treat email addresses with different capitalization as the same address for uniqueness checking.

For example, User@gmail.com and user@gmail.com should not create two separate accounts.

This rule is important because the email address is used to identify users and recover accounts. If duplicate accounts are allowed, login and account management may become confusing.

8. Account Status Must Follow Valid Transitions

An account should move through valid states such as registered, active, suspended and deleted.

For example, a deleted account should not become active again through an ordinary login operation.

This rule is important because it maintains the correct account lifecycle. If invalid state changes are allowed, the system may create contradictory account information.

Therefore, maintaining these invariants ensures that user accounts remain valid, secure and consistent.

Q2. Entities and Value Objects (6 Marks)

In object-oriented domain modelling, domain concepts are classified into two main categories: Entities and Value Objects.

An Entity is an object that has a unique identity and can change its properties during its lifetime without losing its identity.

A Value Object is an object that is identified by its values rather than a unique identity. It is generally immutable.

The classification of the given concepts is explained below.

1. User Account – Entity

A user account is an entity because every account has a unique identity. It contains information such as email address, contact details and account status.

Its properties may change during its lifetime, such as when the user changes their email address or account status.

However, its identity remains the same.

2. User Identifier – Value Object

A user identifier is a value object because it represents the unique identification value assigned to a user account.

It is generally immutable and is compared based on its value.

For example, two user identifiers are equal if they contain the same identifier value.

3. Email Address – Value Object

An email address is a value object because it is identified by its actual value rather than a separate identity.

It should be validated before being accepted and should be stored in a normalized format for consistent comparison.

If the email address changes, a new Email Address value object can be created.

4. Phone Number – Value Object

A phone number is a value object because it represents a contact number and does not require a separate identity.

It should follow a standard format, including the country code.

Two phone number objects are equal if they represent the same normalized phone number.

5. Postal Address – Value Object

A postal address is a value object because it represents a collection of address details, such as street, city, postal code and country.

It does not require a separate identity. Two postal addresses are equal when their relevant address values are the same.

If the user changes their address, a new value object can be created.

6. Verification Token – Entity

A verification token can be modelled as an entity because it has a unique token identifier and a lifecycle.

It contains information such as token value, creation time, expiry time and usage status.

A token may move through states such as active, expired and consumed.

Therefore, it is useful to track each token independently.

Classification Summary

Domain Concept	Classification	Reason
User Account	Entity	Unique identity and lifecycle
User Identifier	Value Object	Identified by its value
Email Address	Value Object	Identified by email value
Phone Number	Value Object	Identified by phone number
Postal Address	Value Object	Identified by address details
Verification Token	Entity	Unique identity and lifecycle

Thus, entities maintain identity throughout their lifetime, whereas value objects represent information based on their values.

Q3. Making Illegal States Unrepresentable (8 Marks)

Making illegal states unrepresentable means designing objects in such a way that invalid combinations of data cannot be created easily.

In the existing User class, information is stored using primitive data types, nullable fields and Boolean flags. This can create several problems.

Problems with the Existing Design

1. Email Address Problem

If the email is stored as a normal string, the system may accept incorrectly formatted email addresses.

2. Email Verification Problem

Using separate Boolean and nullable fields such as emailVerified and verifiedAt may create contradictory states.

For example, the system may have emailVerified = true but verifiedAt = null.

This is an invalid state because a verified email must have a verification date.

3. Verification Token Problem

If the verification token and expiry date are maintained separately, the system may accept an expired token or create a token without a valid expiry date.

4. Phone Number Problem

If the phone number is stored as a normal string, users may enter different formats for the same number. This makes validation and comparison difficult.

5. Postal Address Problem

If street, city, postal code and country are stored independently, some fields may be missing.

For example, an address may contain a street name but no city or postal code.

6. Account Status Problem

If account status is maintained using multiple Boolean flags, contradictory combinations may occur.

For example, an account may be marked as both active and suspended.

Therefore, using primitive fields alone makes it difficult to maintain the correctness of the object.

Improved Object-Oriented Design

To solve these problems, the application should use domain-specific types, encapsulation and validation.

1. User Account Class – Entity

The User Account should be the main entity of the application.

It should contain:

- User Identifier
- Email Address
- Email Verification Status
- Optional Phone Number
- Optional Postal Address
- Account Status

All important data members should be private.

The user account should provide controlled methods for registration, email verification, updating contact details, suspension and deletion.

2. Email Address – Value Object

Instead of storing an email address as a normal string, the system should use a separate Email Address value object.

It should:

- Validate the email format.
- Reject empty or invalid email addresses.
- Store the normalized email value.
- Prevent direct modification after creation.

This ensures that invalid email addresses cannot enter the system.

3. Phone Number – Value Object

Phone Number should be represented as a separate value object.

It should:

- Include the country code.
- Follow a standard format.
- Reject invalid phone numbers.
- Be optional for users who do not provide a phone number.

This ensures consistent phone number representation.

4. Postal Address – Value Object

Postal Address should contain:

- Street
- City
- Postal Code
- Country

The object should validate the required fields before accepting an address.

This prevents incomplete postal addresses from being stored.

5. Verification Token – Separate Domain Object

Verification tokens should be maintained separately with the following information:

- Unique Token ID
- Token Value
- Creation Date and Time
- Expiry Date and Time
- Token Status

The token should be accepted only if it is valid, unused and not expired.

Suitable Representation for Email Verification

Instead of using separate Boolean and nullable fields, the application should use a single verification state.

The verification state can contain:

1. UNVERIFIED: The user has registered but has not verified their email.
2. VERIFIED: The user has successfully completed email verification.
3. PENDING: A verification request has been generated and is waiting for completion.

The verification date should be associated with the VERIFIED state rather than maintained as an unrelated nullable field.

Suitable Representation for Optional Contact Information

Phone number and postal address should be optional value objects.

If the user provides a phone number or postal address, the system should validate it before storing it.

If the user does not provide these details, the system should represent them as absent rather than creating incomplete objects.

Improved Object Structure (Textual UML Representation)

The proposed design can be represented as follows:

User Account (Entity)

- User Identifier
- Email Address
- Email Verification

- Phone Number (Optional)
- Postal Address (Optional)
- Account Status

Email Address (Value Object)

- Email Value
- Normalized Email

Email Verification

- Verification State
- Verification Date (when verified)

Verification Token (Entity)

- Token ID
- Token Value
- Created At
- Expiry Date
- Token Status

Phone Number (Value Object)

- Country Code
- Phone Number

Postal Address (Value Object)

- Street
- City
- Postal Code
- Country

Account Status

- REGISTERED
- ACTIVE
- SUSPENDED
- DELETED

How the Improved Design Prevents Illegal States

- Invalid email addresses are rejected during object creation.
- A verified email must have a verification date.
- Expired verification tokens cannot be accepted.
- Phone numbers must follow a standard format.
- Incomplete postal addresses are rejected when an address is provided.
- Suspended and deleted accounts cannot be used for login.
- Private fields and controlled methods prevent direct modification of account information.

This follows the concepts of encapsulation, validation and valid object states explained in our Session-1 notes.

Therefore, the improved design reduces invalid states and makes the application easier to maintain.

Q4. Email Verification and Account Lifecycle (6 Marks)

Email verification is an important process in user account management. It ensures that the registered email address actually belongs to the user.

The application should follow a proper workflow during registration and verification.

1. Information Maintained During User Registration

When a user registers, the system should maintain:

- Unique User ID.
- Email Address.
- Password information.
- Registration Date and Time.
- Email Verification Status.
- Optional Phone Number.
- Optional Postal Address.
- Account Status.

Initially, the email verification status should be UNVERIFIED and the account should not be treated as fully active.

2. Generation and Validation of Verification Tokens

After registration, the system should generate a unique and secure verification token.

The token should have:

- Unique token value.
- Creation date and time.
- Expiry date and time.
- Usage status.

The application sends the verification link to the registered email address.

When the user clicks the link, the system checks whether the token exists, is valid and has not expired or already been used.

If all conditions are satisfied, the email is marked as verified and the verification date is recorded.

3. Handling Expired Verification Tokens

If the user tries to verify their email using an expired token, the system should reject the request.

The application should not mark the email as verified.

Instead, the user should be allowed to request a new verification token.

The old token should be marked as expired or invalid so that it cannot be reused.

4. When a Verified User Changes Their Email Address

If a user changes their registered email address, the new email address should be treated as unverified.

The application should:

1. Validate the new email address.
2. Check whether the new email address is already registered.
3. Store the new email as pending verification.
4. Generate a new verification token.
5. Send a verification link to the new email address.
6. Mark the new email as verified only after successful verification.

The old verified email should not automatically make the new email verified.

5. Preventing Suspended and Deleted Accounts from Logging In

The application should check the account status whenever a user tries to log in.

- REGISTERED: The user has registered but has not completed email verification.
- ACTIVE: The user can log in normally.
- SUSPENDED: Login should be rejected until the account is restored by an authorized person.
- DELETED: Login and normal account access should not be permitted.

This ensures that users with suspended or deleted accounts cannot access the application.

6. Preventing Contradictory Verification States

The application should use a dedicated Email Verification type instead of separate Boolean and nullable fields.

For example, the system should not allow an email to be marked as verified without recording its verification date.

The verification state should be changed only through controlled methods.

The application should also maintain valid state transitions:

UNVERIFIED → VERIFIED

When the email address changes:

VERIFIED → UNVERIFIED (for the new email address)

This ensures that the verification process remains consistent and prevents contradictory states.

Therefore, a properly designed verification workflow improves account security and maintains correct user information.

Q5. Cross-Aggregate Rules and Design Evaluation (4 Marks)

Email uniqueness is an important business rule in a user account management system.

It ensures that the same email address cannot be used to create multiple accounts.

1. Is Email Uniqueness an Invariant of an Individual User Object?

No. Email uniqueness is not an invariant of an individual User object.

An individual User object can validate whether its email address is correctly formatted. However, it cannot independently check whether another user is already using the same email address.

Email uniqueness depends on comparing the email address with all other registered users.

Therefore, it is a cross-aggregate business rule.

2. How Should the Application Enforce Email Uniqueness?

The application should maintain a centralized user registration service.

Whenever a new user registers, the service should:

1. Validate the email address.
2. Normalize the email address for consistent comparison.
3. Check whether the email address already exists.
4. Reject registration if another account is using the same email.
5. Create the new user account only if the email address is available.

This prevents duplicate accounts during normal registration.

3. What Happens If Two Users Register with the Same Email Simultaneously?

Suppose two users try to register using the same email address at the same time.

Both registration requests may check the database and find that the email address is available.

As a result, both requests may attempt to create accounts with the same email address.

This is called a race condition.

It may create duplicate accounts and cause login or account recovery problems.

4. What Additional Protection Should Be Provided by the Database?

The database should enforce a UNIQUE constraint on the normalized email address.

This ensures that even if two registration requests occur simultaneously, only one account can be created with that email address.

The database should reject the second registration attempt.

This provides an additional level of protection beyond application-level validation.

5. One Advantage and One Disadvantage of Strongly Typed Verification

Advantage:

Strongly typed verification makes invalid states difficult to create. It ensures that verification information is maintained in a controlled manner and prevents contradictory combinations of Boolean and nullable fields.

Disadvantage:

It requires additional classes and design effort. Developers must define and maintain separate domain types and state transitions, which may increase the initial complexity of the application.

Therefore, strongly typed verification improves data correctness and maintainability, even though it requires more design effort.